

Agent Foundations — Write-Up

Assignment: Week 2 Day 1 — Reasoning Loops, Tool Calling & Raw Python Agents **Model:** `gemini-2.5-flash` (via `google-genai` SDK)

1. The ReAct Loop

ReAct = **Reason** → **Act** → **Observe**, repeated until the model has enough information to answer.

```
loop:
    response = model(history, tools)
    if response has function_call:
        result = execute_tool(response.function_call)
        history.append(function_call, function_result)
    else:
        return response.text
```

At each iteration the model first *reasons* about what it still needs, then *acts* by emitting a `function_call` (name + arguments matching a JSON schema), and finally *observes* the `function_result` we send back before deciding whether to reason again or return a final answer. A `max_iterations` cap prevents the loop from running forever if the model keeps calling tools without converging.

2. Tool Schemas Used

calculator — `operation` (enum: add/subtract/multiply/divide), `a` (number), `b` (number). Both operands and the operation are marked `required` so the model can't call it half-specified.

`get_weather` — `location` (string, city name), `required`. Backed by a small mock lookup table rather than a real network call, per the assignment's "stub" suggestion.

Both schemas follow the same shape: `name`, a task-oriented `description`, and a `parameters` object with typed fields and a `required` list. The descriptions are written to say *when* to use the tool ("use this whenever the user asks for a numeric computation...") rather than just *what* it does — this is what lets Gemini pick the right tool and fill in correctly typed arguments instead of guessing or hallucinating.

3. Failure Modes Observed (with Mitigations)

Failure Mode	What Happened	Mitigation
Infinite loops	Without a cap, a confused agent could keep calling tools indefinitely	<code>max_iterations</code> hard stop in the loop
Hallucinated tool calls	Model occasionally names a tool that was never declared (e.g. <code>send_email</code>)	Validate <code>function_call.name</code> against the registered tool dict before executing; return a structured error if unknown
Wrong / missing tool arguments	Malformed or missing fields would crash a naive <code>fn(**args)</code> call	Wrap execution in <code>try/except</code> ; schema's <code>required</code> list narrows what the model is likely to omit
Silent errors inside a tool	A tool exception (e.g. divide by zero) would otherwise kill the whole run	Wrap tool execution in <code>try/except</code> , return the error as the <code>function_result</code> so the model can react instead of crashing

Model skips the tool when it should call it	For under-specified prompts the model sometimes answers directly instead of using an available tool	Sharper tool <code>description</code> , or a system instruction nudging tool use for matching queries
Tool timeout / hang	Not triggered by our mock tools, but a real network-backed tool could hang	Wrap the call with a per-tool timeout and return an error observation on expiry

Three deliberate break tests: an **ambiguous request** ("Is it nice out?") led the model to ask a clarifying question rather than call a tool blindly; a **tool-error case** (divide by zero) surfaced as an `{"error": ...}` observation that the model explained back to the user instead of crashing on; and a **missing-tool case** ("send an email...") was caught by the loop's unknown-tool-name guard and reported as a limitation rather than a fabricated success.

4. Why Frameworks Like LangChain / LangGraph / CrewAI Exist

Building this loop by hand makes the value of these frameworks concrete: they package the exact plumbing above — history/state management, schema validation, retries, streaming, memory backends, and multi-agent coordination — so it isn't rewritten per project. The cost is that they abstract away the very mechanics (`function_call` → `execute` → `function_result` → `repeat`) this exercise was meant to expose. Having built the raw version first makes it much easier to reason about what a framework is actually doing when something breaks.